

AI Background - Continued

Lecture 04

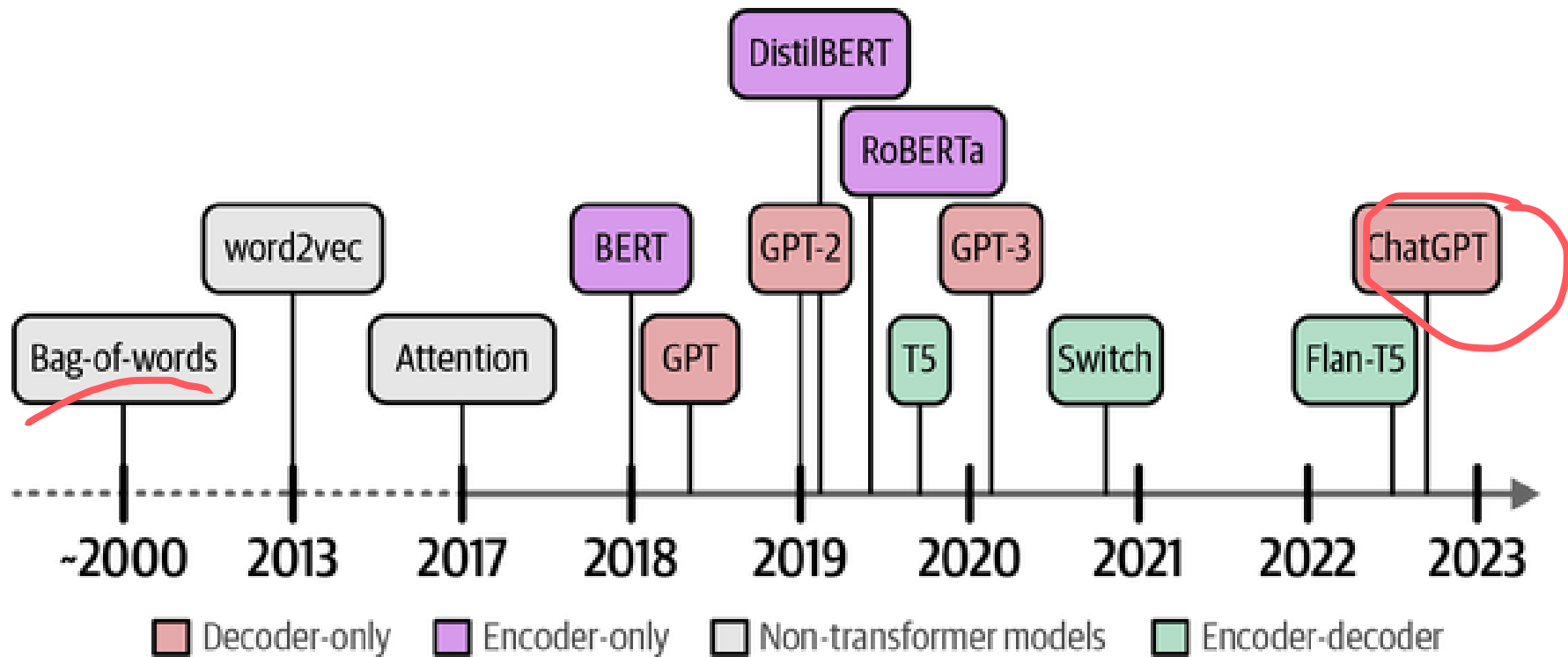
Review developments leading to Language AI

Overview / outline of current topics

- Language Tasks Background
- History of Language AI (2000-2023)
- Neural Networks Background
 - Overview of main concepts
 - AI developments leading to ChatGPT (2023)
- History of Agents (2023-2026)
 - Summarize key accomplishments
 - Interpret what has happened
 - So we can start to understand key moments
- Then we are ready to understand security issues (2 classes from now!)

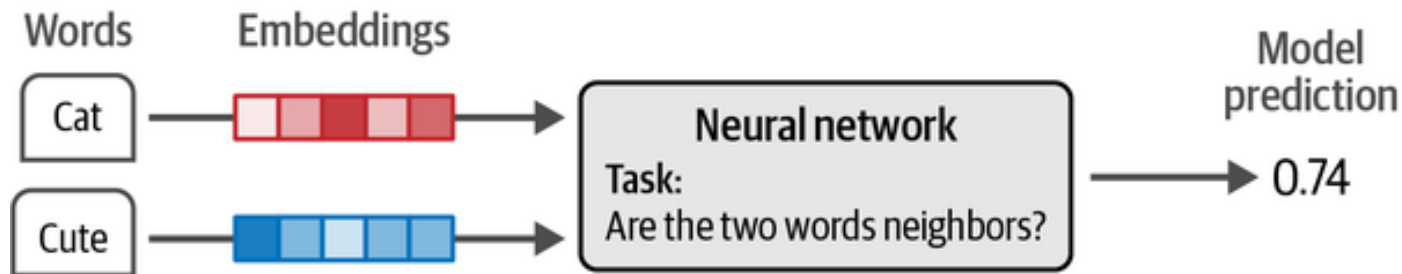
History

Important Milestones in the History of Language AI

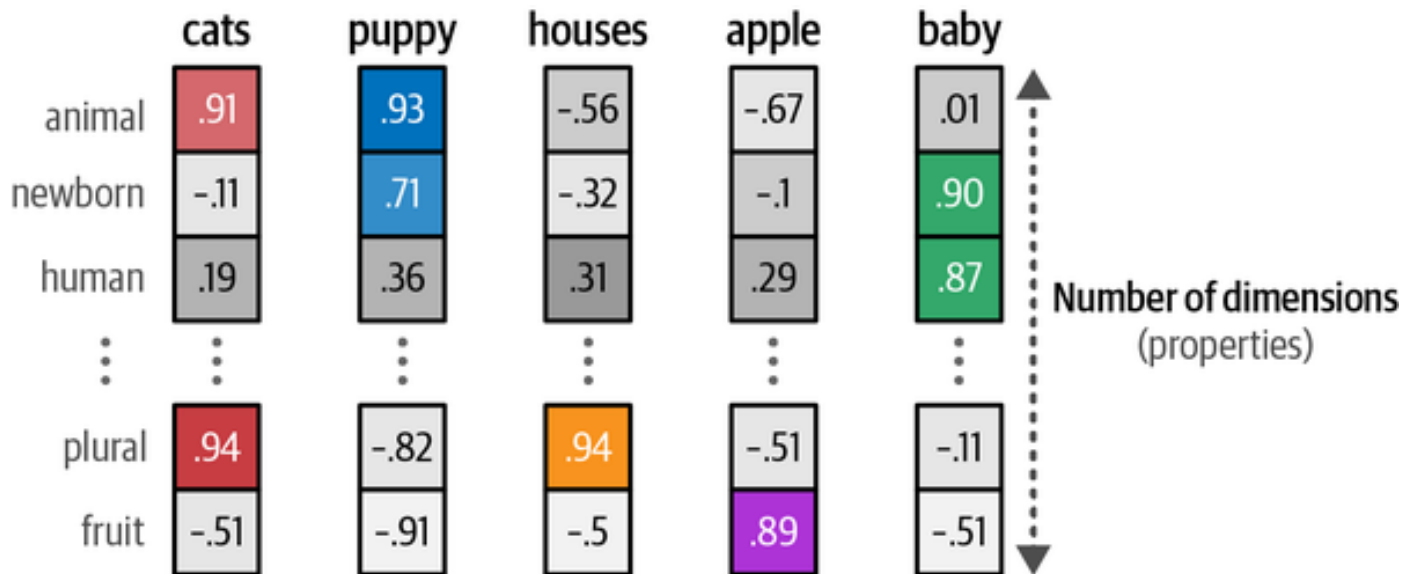


- Bag-of-words, although an elegant approach, has a flaw: it treats language as an almost literal *bag* of words, ignoring the semantic nature – the meaning – of text
- **word2vec** (2013) was one of the first successful attempts at capturing meaning in **embeddings**
 - Embeddings are vector representations of data that attempt to capture its meaning
- Word2vec learns semantic representations of words by training on vast amounts of text, like the entirety of Wikipedia
- To generate these semantic representations, word2vec leverages neural networks.

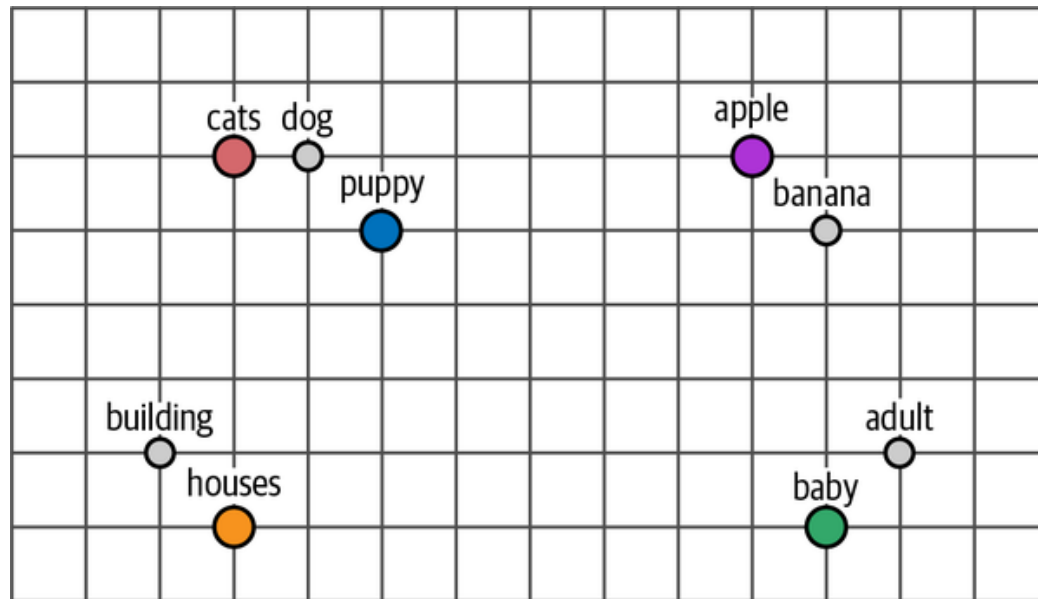
Word2Vec: Embeddings



Word2Vec: Embedding Dimensions



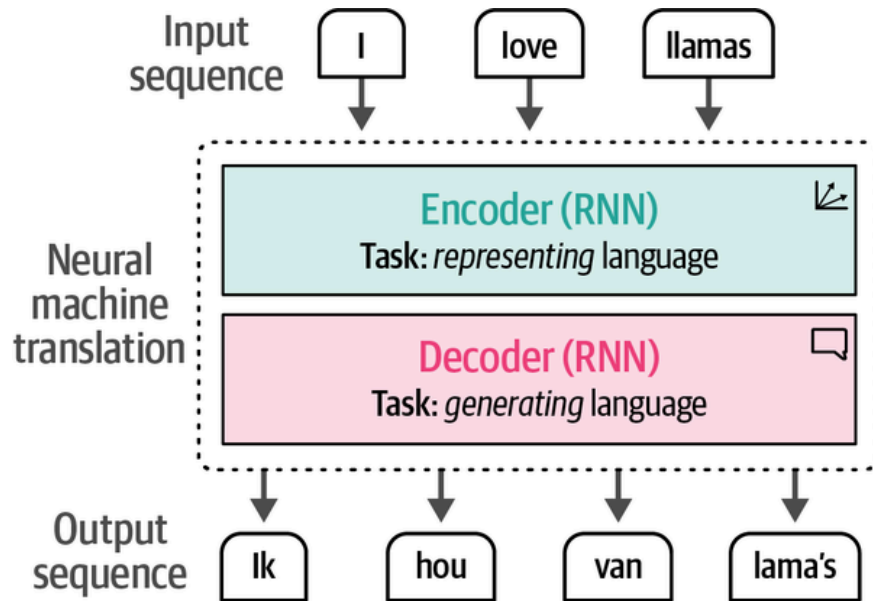
Word2Vec: Embedding Similarities



Types of Embeddings

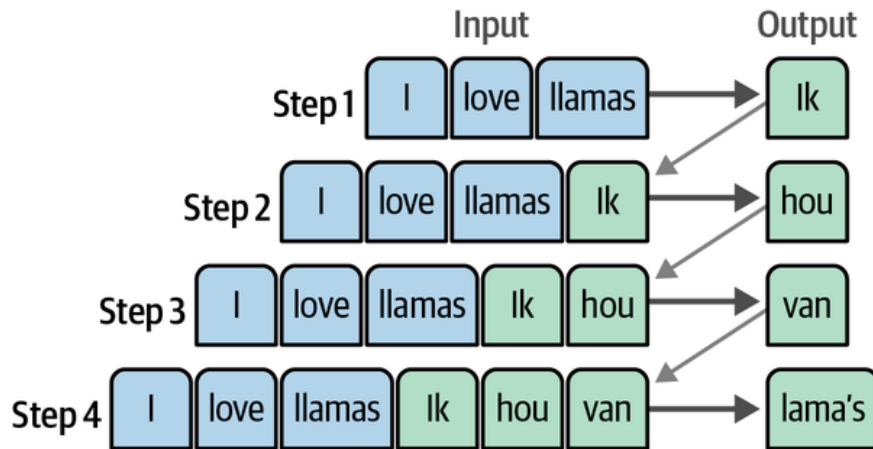
From Static to Contextual: RNNs

- Word2vec embeddings are **static**: a word like "bank" gets one fixed embedding
- However, meaning depends on **context**
- **Recurrent neural networks (RNNs)** were a key step toward context-aware representations, by modeling a sentence as a *sequence*
- RNNs are used for two tasks:
 - **Encoding**: representing an input sentence
 - **Decoding**: generating an output sentence
- Example: encoding "I love llamas" and decoding it into Dutch, "Ik hou van lama's"



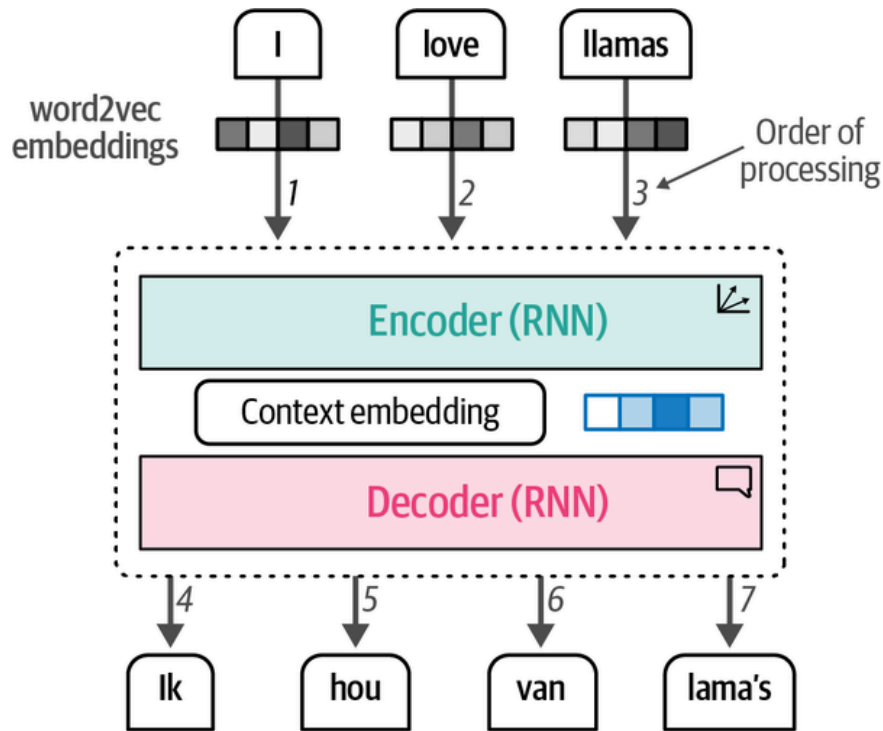
Autoregressive Steps

- Each step in this architecture is **autoregressive**
 - a model is autoregressive if it specifies output variables that are dependent on their own previous values
- When generating the next word, this architecture needs to consume all previously generated words



Context Embedding

- The encoding step aims to represent the input as well as possible, producing a **context embedding** that serves as input to the decoder
- Its inputs are word embeddings, so we can use word2vec for the initial representations
- Inputs are processed **sequentially**, one at a time – as is the output
- This single context embedding makes it difficult to handle **longer sentences**, since one embedding must represent the entire input



Attention

- In 2014, a solution called **attention** was introduced that highly improved upon the original architecture
- Attention allows a model to focus on parts of the input sequence that are relevant to one another – to "**attend**" to each other – and amplify their signal
- Paper: Bahdanau et al. (2014),
arxiv.org/pdf/1409.0473

Published as a conference paper at ICLR 2015

NEURAL MACHINE TRANSLATION BY JOINTLY LEARNING TO ALIGN AND TRANSLATE

Dzmitry Bahdanau
Jacobs University Bremen, Germany

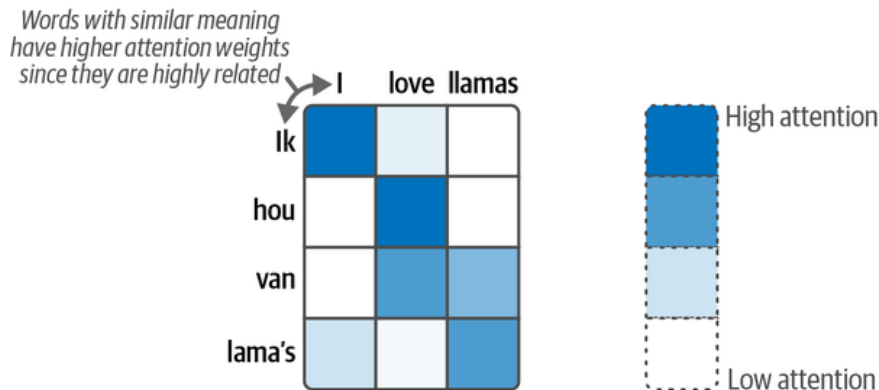
KyungHyun Cho Yoshua Bengio*
Université de Montréal

ABSTRACT

Neural machine translation is a recently proposed approach to machine translation. Unlike the traditional statistical machine translation, the neural machine translation aims at building a single neural network that can be jointly tuned to maximize the translation performance. The models proposed recently for neural machine translation often belong to a family of encoder-decoders and encode a source sentence into a fixed-length vector from which a decoder generates a translation. In this paper, we conjecture that the use of a fixed-length vector is a bottleneck in improving the performance of this basic encoder-decoder architecture, and propose to extend this by allowing a model to automatically (soft-)search for parts of a source sentence that are relevant to predicting a target word, without having to form these parts as a hard segment explicitly. With this new approach, we achieve a translation performance comparable to the existing state-of-the-art phrase-based system on the task of English-to-French translation. Furthermore, qualitative analysis reveals that the (soft-)alignments found by the model agree well with our intuition.

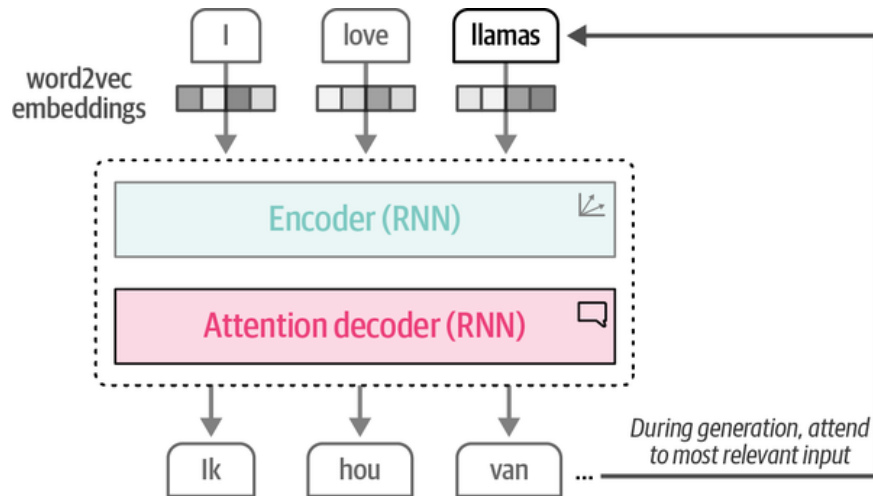
Attention Example

- The output word "lama's" is Dutch for "llamas," which is why the attention between both is **high**
- Similarly, the words "lama's" and "I" have **lower** attention since they aren't as related
- Attention allows a model to "attend" to certain parts of sequences that might relate more or less to one another



Attention Decoding

- Adding attention to decoder lets the RNN generate a signal for each input word related to the potential output
- Instead of passing only context embedding, the **hidden states of all input words** are passed to the decoder
- After generating "lk," "hou," and "van," attention lets the decoder focus on "llamas" before generating "lama's"
- Compared to word2vec, this architecture represents the sequential nature of text and its context by "attending" to the entire sentence



This sequential nature **precludes parallelization** during training

"Attention Is All You Need"

- The true power of attention – and what drives the amazing abilities of large language models – was first explored in the well-known "**Attention Is All You Need**" paper, released in 2017
- The authors proposed a network architecture called the **Transformer**, based solely on the attention mechanism, removing the recurrence network we saw previously
- Compared to the recurrence network, the Transformer could be trained **in parallel**, tremendously speeding up training
- Paper: Vaswani et al. (2017),
arxiv.org/abs/1706.03762

Attention Is All You Need

Ashish Vaswani*
Google Brain
avaswani@google.com

Noam Shazeer*
Google Brain
noam@google.com

Niki Parmar*
Google Research
nikip@google.com

Jakob Uszkoreit*
Google Research
usz@google.com

Llion Jones*
Google Research
llion@google.com

Aidan N. Gomez* †
University of Toronto
aidan@cs.toronto.edu

Lukasz Kaiser*
Google Brain
lukaszkaizer@google.com

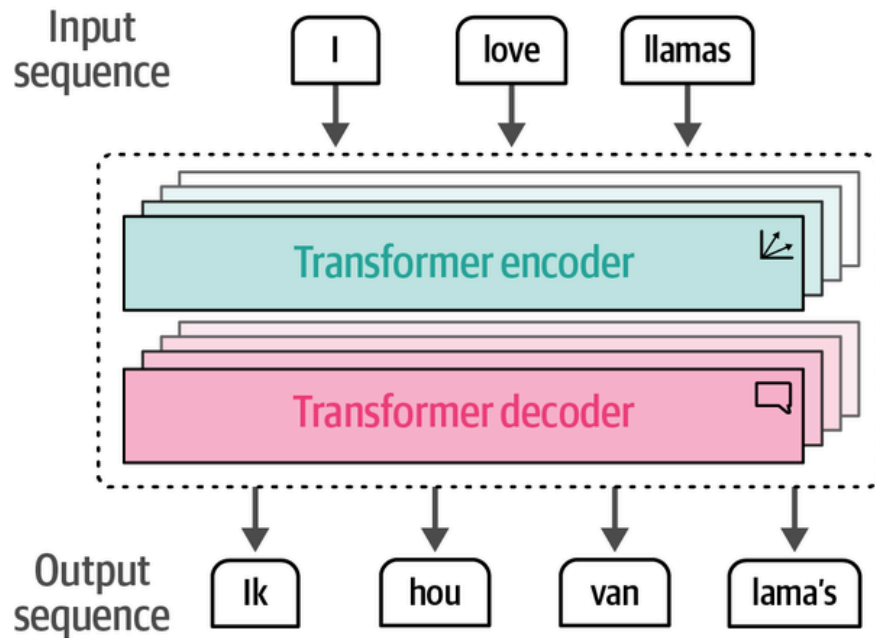
Illia Polosukhin* ‡
illia.polosukhin@gmail.com

Abstract

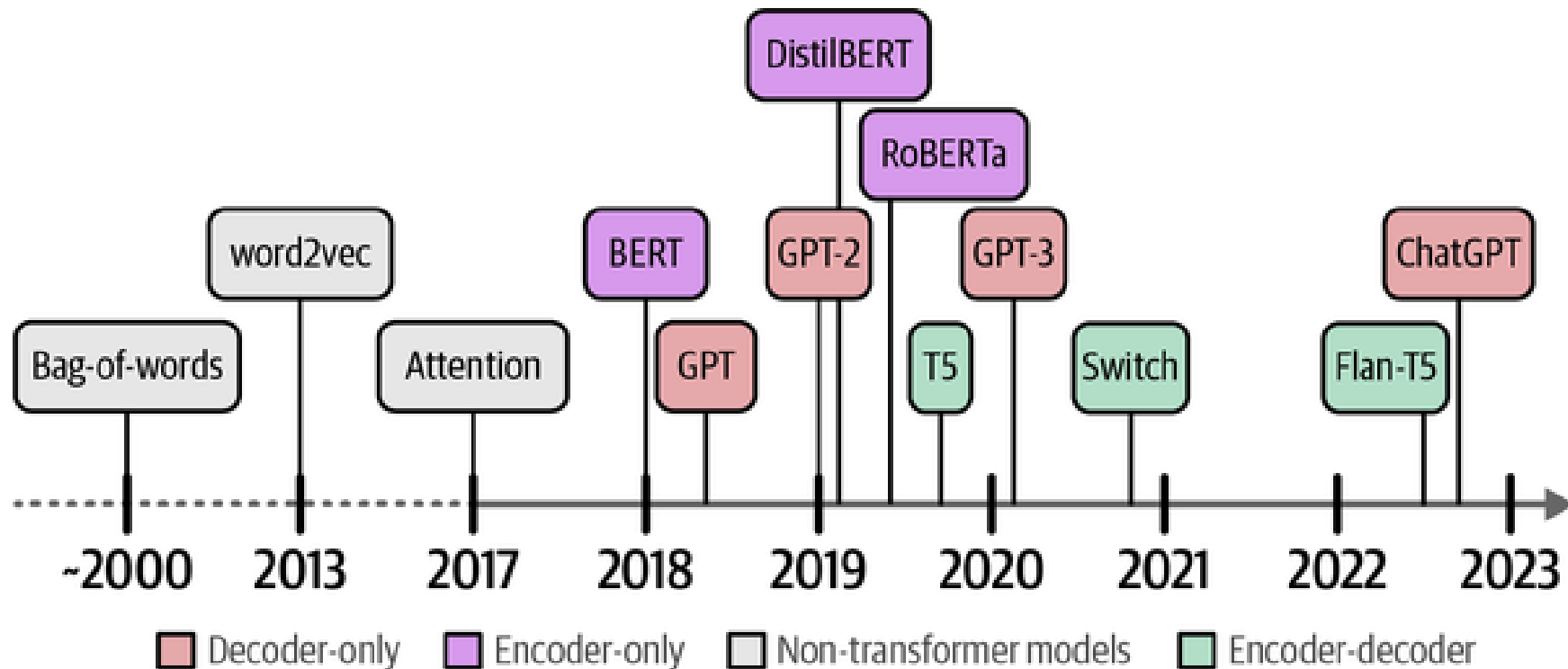
The dominant sequence transduction models are based on complex recurrent or convolutional neural networks that include an encoder and a decoder. The best performing models also connect the encoder and decoder through an attention mechanism. We propose a new simple network architecture, the Transformer, based solely on attention mechanisms, dispensing with recurrence and convolutions entirely. Experiments on two machine translation tasks show these models to be superior in quality while being more parallelizable and requiring significantly less time to train. Our model achieves 28.4 BLEU on the WMT 2014 English-to-German translation task, improving over the existing best results, including ensembles, by over 2 BLEU. On the WMT 2014 English-to-French translation task, our model establishes a new single-model state-of-the-art BLEU score of 41.8 after training for 3.5 days on eight GPUs, a small fraction of the training costs of the best models from the literature. We show that the Transformer generalizes well to other tasks by applying it successfully to English constituency parsing both with large and limited training data.

Transformer Architecture

- In the Transformer, encoding and decoding components are **stacked** on top of each other
- This architecture remains **autoregressive**, needing to consume each generated word before creating a new word
- Now, both the encoder and decoder blocks revolve around **attention**, instead of leveraging an RNN with attention features

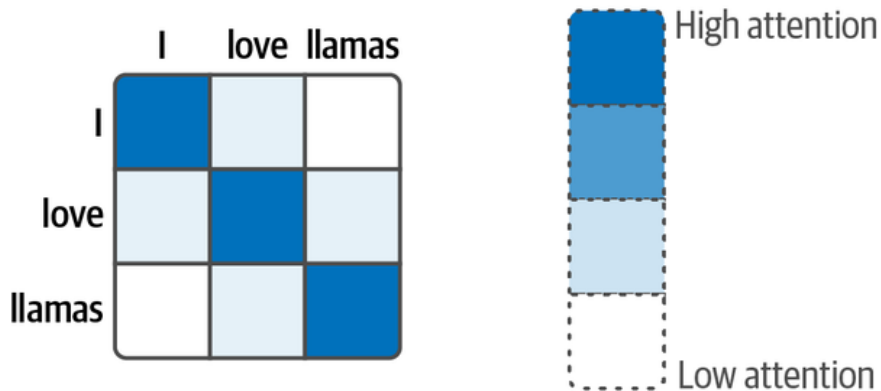


Important Milestones in the History of Language AI



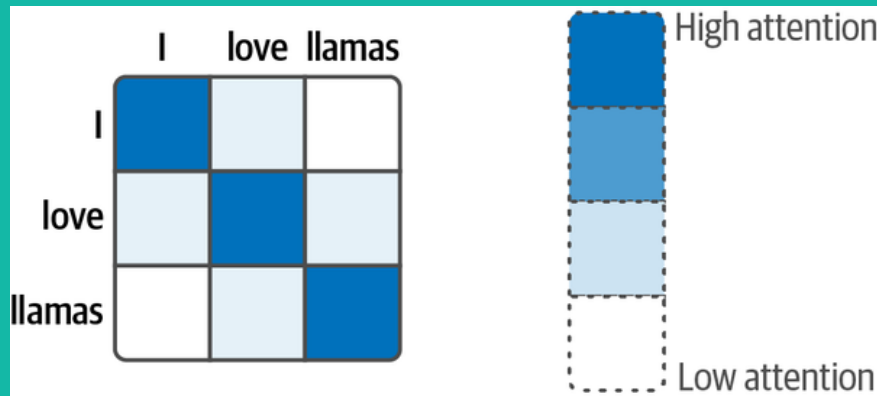
Transformer Encoder

- The encoder block in the Transformer consists of two parts: **self-attention** and a **feedforward neural network**
- Compared to previous methods of attention, self-attention can attend to different positions **within a single sequence**, more easily and accurately representing the input sequence
- Instead of processing one token at a time, it can look at the **entire sequence in one go**



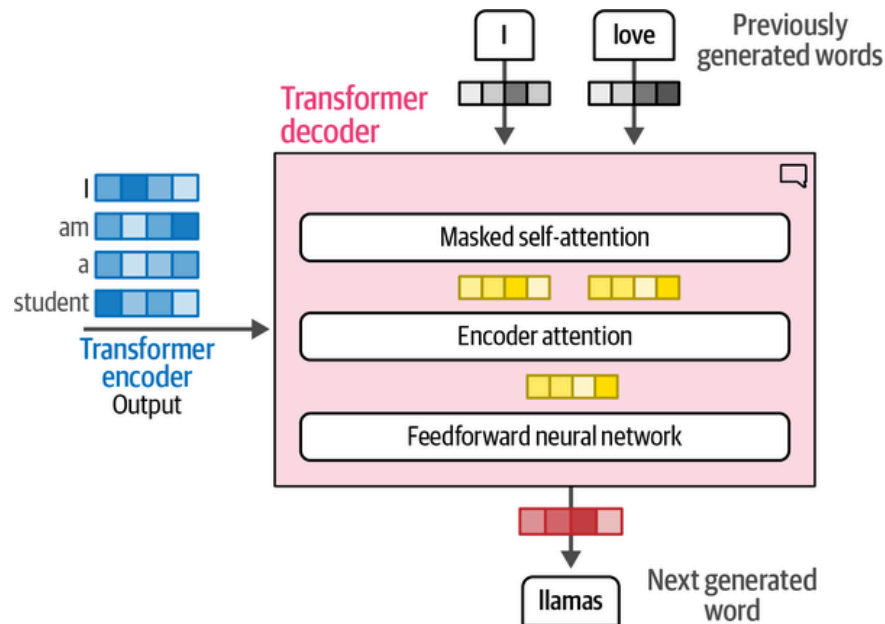
Self-Attention

- Compared to previous methods of attention, self-attention can attend to different positions **within a single sequence**, more easily and accurately representing the input sequence
- Instead of processing one token at a time, it can be used to look at the **entire sequence in one go**



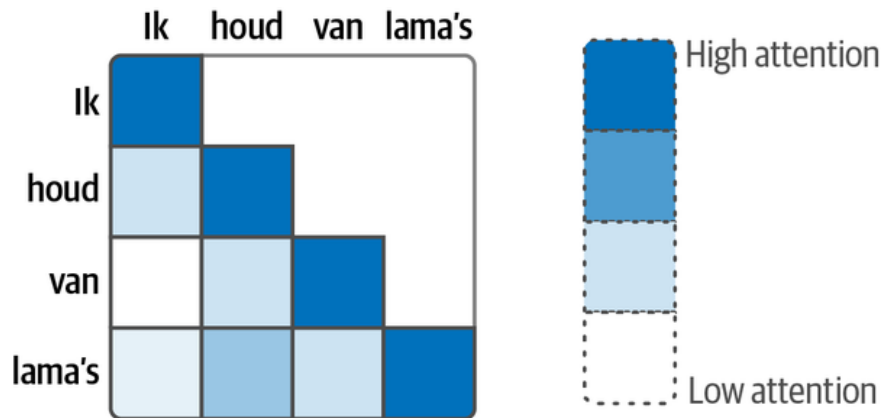
Attention Decoder

- Compared to the encoder, the decoder has an additional layer that pays attention to the **output of the encoder**, to find the relevant parts of the input
- This process is similar to the RNN attention decoder we discussed previously



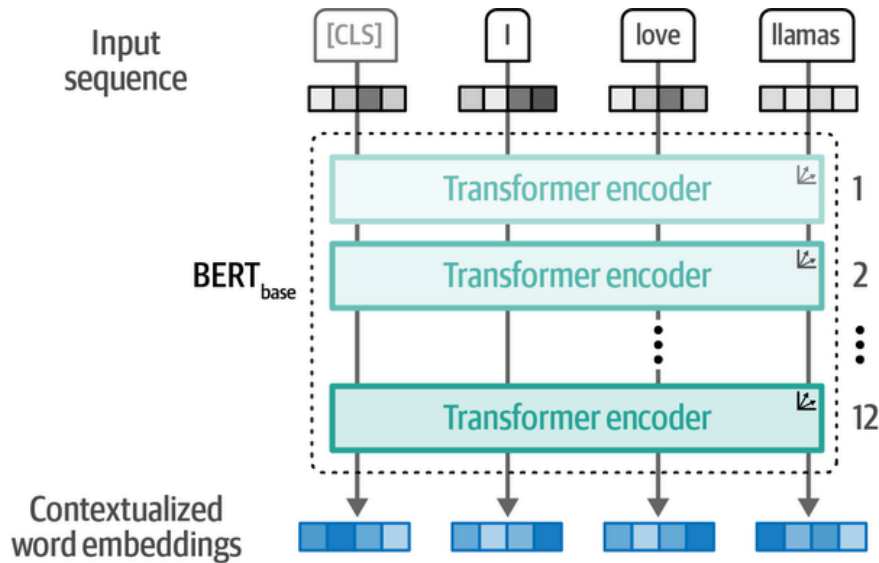
Masked Decoder

- The self-attention layer in the decoder **masks future positions**, so it only attends to earlier positions
- This prevents **leaking information** when generating the output



Representation Models: Encoder-Only Models

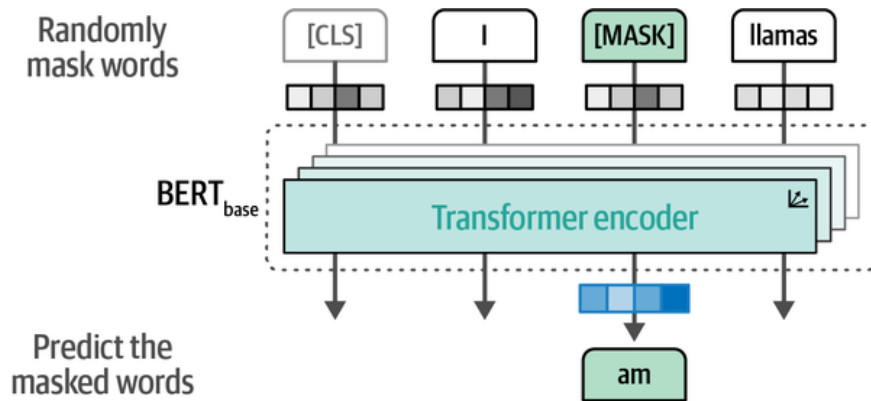
- The original Transformer is an encoder-decoder architecture that serves translation well, but isn't easily used for other tasks like text classification
- In 2018, **BERT** (Bidirectional Encoder Representations from Transformers) was introduced – usable for a wide variety of tasks, and the foundation of Language AI for years to come
- BERT is an **encoder-only** architecture focused on representing language, using the same encoder blocks as before: self-attention followed by a feedforward neural network
- The input contains an additional token, **[CLS]** (classification token), used as the



- We often use the [CLS] token as the input embedding for fine-tuning the model on specific tasks, like classification

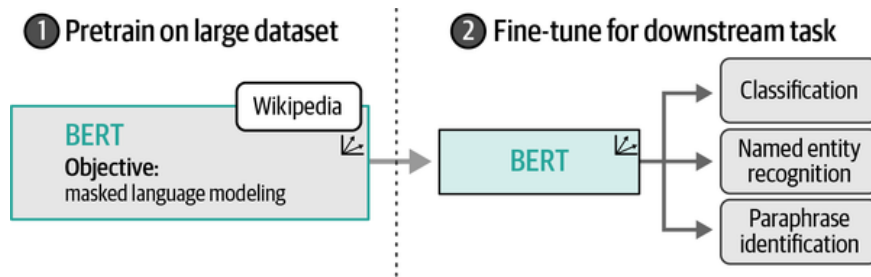
Masked Language Modeling

- Training these encoder stacks is difficult – BERT approaches this with a technique called **masked language modeling**
- This method masks part of the input for the model to predict
- This prediction task is difficult, but allows BERT to create more accurate (intermediate) representations of the input



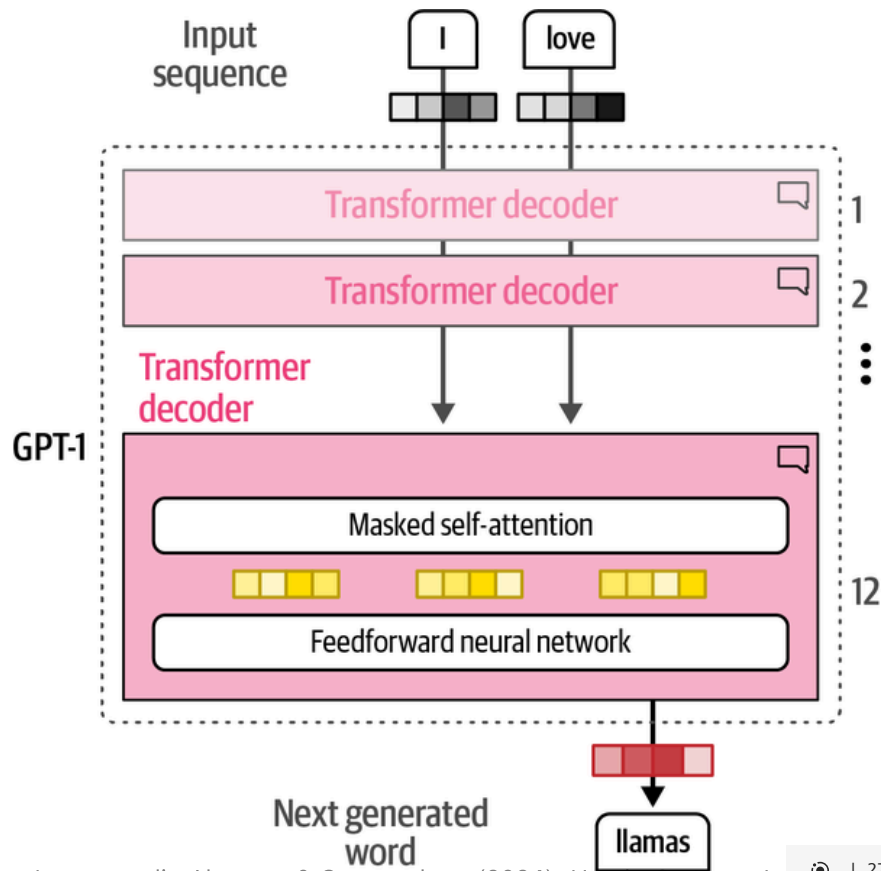
Task: Transfer Learning

- This architecture and training procedure makes BERT and related architectures incredible at representing contextual language
- BERT-like models are commonly used for **transfer learning**: first **pretraining** for language modeling, then **fine-tuning** for a specific task
- Training BERT on the entirety of Wikipedia teaches it to understand the semantic and contextual nature of text
- We can then use that pretrained model to fine-tune it for a specific task, like text classification



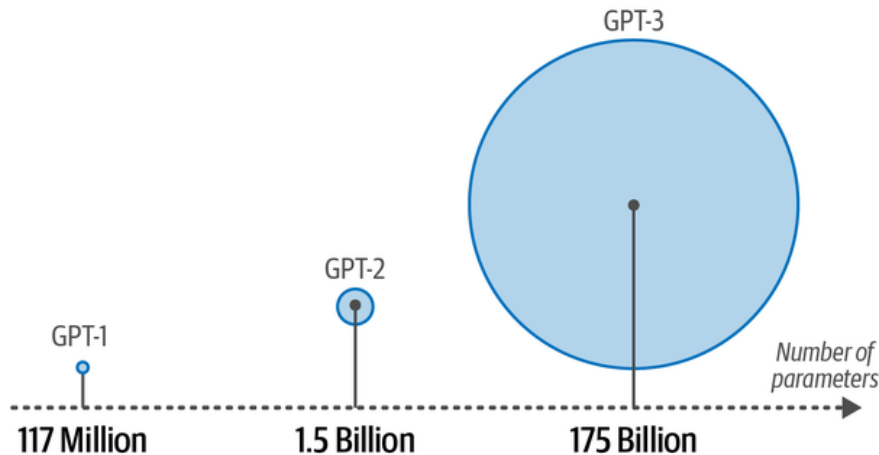
Generative Models: Decoder-Only Models

- 2018: a **decoder-only** architecture was proposed for generative tasks: the **Generative Pre-trained Transformer (GPT)** – now known as **GPT-1**
- It stacks decoder blocks, similar to the encoder-stacked architecture of BERT
- GPT-1 was trained on a corpus of 7,000 books and **Common Crawl**, a large dataset of web pages
- The resulting model had **117 million parameters** – numerical values representing the model's understanding of language
- more parameters to greatly influence a model's capabilities and performance



Bigger Models: GPT-2 and GPT-3

- **GPT-2** had **1.5 billion** parameters, and **GPT-3** quickly followed with **175 billion** parameters
- These generative decoder-only models – especially the "larger" ones – are commonly referred to as **large language models (LLMs)**
- The term LLM isn't reserved only for generative models (decoder-only) – it also covers representation models (encoder-only), like BERT

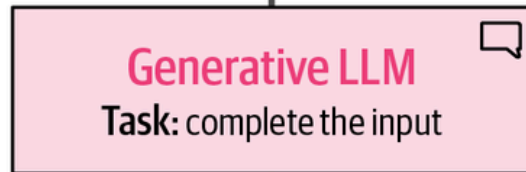


ChatGPT : 2023

- Generative LLMs, as sequence-to-sequence machines, take in text and attempt to **autocomplete** it
- A handy feature – but their true power shone from being trained as a **chatbot**
- Instead of just completing text, what if they could be trained to **answer questions**? By fine-tuning, we get **instruct** or **chat** models that can follow directions
- The resulting model takes in a user query (**prompt**) and outputs a response that would most likely follow it – which is why generative models are often called **completion models**

User query
(prompt)

Tell me something about llamas

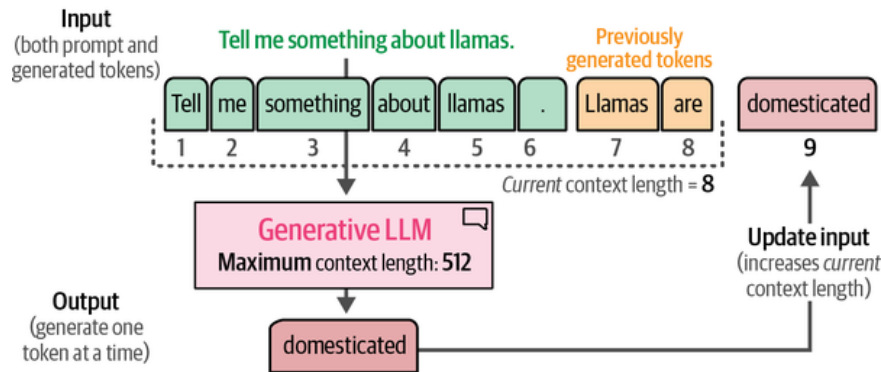


Output
(completion)

Llamas are domesticated South American camelids, widely used as pack animals by Andean cultures since pre-Hispanic times. With their fluffy coat, long neck, and distinctive facial features...

Context in ChatGPT

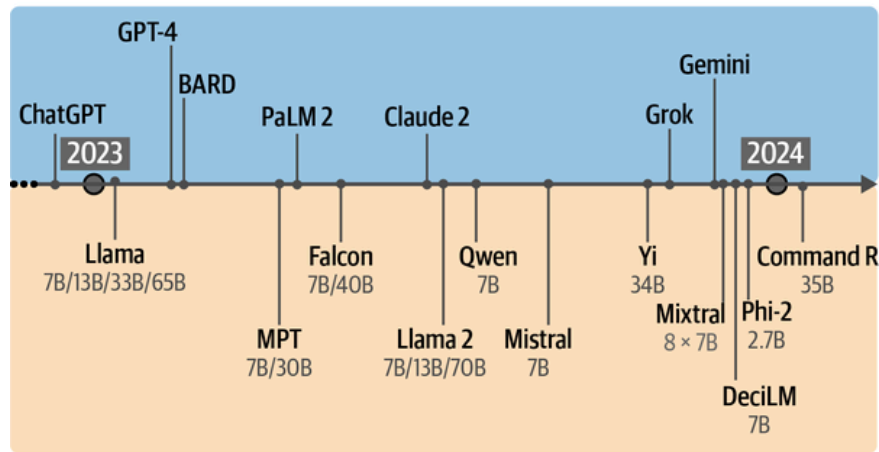
- A vital part of these completion models is the **context length**, or **context window**
- The context length represents the maximum number of tokens the model can process
- A large context window allows entire documents to be passed to the LLM
- Due to the autoregressive nature of these models, the **current** context length increases as new tokens are generated



2023: The Year of GenAI

- LLMs had a tremendous impact on the field, leading some to call 2023 "**The Year of Generative AI**" – driven by the release, adoption, and media coverage of ChatGPT (GPT-3.5)
- "ChatGPT" refers to the **product**, not the underlying model – initially powered by GPT-3.5, now including more performant variants like GPT-4
- GPT-3.5 wasn't the only impactful model: both **open source** and **proprietary** LLMs reached people at an incredible pace
- These open source base models are often called **foundation models**, and can be fine-

Proprietary models



Open models

